Search and Automated Planning

LiangKun Chen and GuanDu Zhao
lc2108@hw.ac.uk and gz2007@hw.ac.uk
Heriot-Watt University - OUC CW1 11

Abstract

This is an implementation of the F29AI coursework 1(Heriot-Watt University, 2025) of the group 11. The project consists of two independent projects: a Sudoku Solver with GUI and a Lunar Mission Planner using PDDL to plan. Our code and video can be found at https://github.com/KunLiangChen/F29AI-CourseWork1 and our video can be found in Onedrive

Keywords – AI, Sudoku, PDDL, lunar mission

1 Search Algorithm on Sudoku

In this part, we solve and analyze the Sudoku with search algorithms. This section includes two subsections: Part 1A and Part 1B. Part1A detail the formal Constraint Satisfaction Problem(CSP) definition and discuss the time complexity of brute-force search vs backtracking in Sudoku. Part 1B designs and implement an intelligent system in Python that can efficiently solve any valid 9x9 Sudoku puzzle.

1.1 Part 1A

In this subsection, we will give out a formal CSP definition and discuss the complexity of two search algorithms. First, we describe this problem with natural language. A Sudoku is as follows:

				7				6
			9				4	
7		5		3				1
	6		1					5
8								3
3					4		2	
4				2		3		7
	2				6			
5				9				

© sudoku.today

Figure 1: Sudoku.

This is a 9×9 grid, and the objective is to fill the grid with digits in such a way that each row, each column, and each of the 9 principal 3×3 sub-squares contains all of the digits from 1 to 9.

A Constraint Satisfaction Problem (CSP)[1] is defined as a triple:

$$\text{CSP} = (X, D, C). \tag{1}$$

Where X is a set of variables, D is a set of domains, and C is a set of constraints. Refer to former work[2], we can give out a formal CSP definition for this Sudoku.

$$X = \{X_{i,j} | 1 \leq i, j \leq 9\}. \tag{2}$$

$$D = \begin{cases} \{1, 2, \ldots, 9\}, & \text{if}(i,j) \text{ is empty;} \\ \{v_{i,j}\}, & \text{if}(i,j) \text{ has a given clue } v_{i,j}. \end{cases} \tag{3}$$

$$C = \{X_a \neq X_b | a \sim b\}. \tag{4}$$

Where $X_{i,j}$ denotes a variable waiting to be assigned at i line and j column. $a \sim b$ denotes that cells a and b are in the same row, column, or subgrid. There are two basic algorithms.

The brute-force search (also known as uninformed search) systematically explores all possible states without using domain knowledge[3]. The backtracking algorithm incrementally builds candidates for the solution and abandons a candidate ("backtracks") as soon as it determines that it cannot lead to a valid solution[4].

The brute-force approach considers every possible assignment of values to the Sudoku grid without regard for constraints until a valid configuration is found. If there are n empty cells, and each cell can take any of the 9 digits, the total number of possible assignments is 9^n. Therefore, the worst-case time complexity is:

$$T_{\text{brute}} = O(9^n), \tag{5}$$

which is computationally infeasible even for moderately incomplete Sudoku puzzles. This method does not exploit any problem structure or pruning, so the entire search space must be explored.

Backtracking, by contrast, applies a depth-first search strategy with constraint propagation. At each step, the algorithm assigns a value only if it satisfies all Sudoku constraints (row, column, and block). When a partial assignment leads to a violation, the algorithm backtracks—removing the last value and trying an alternative. This pruning greatly reduces the effective search space.

While the theoretical worst-case time complexity of backtracking is also exponential:

$$T_{\text{backtrack}} = O(9^n), \tag{6}$$

The practical runtime is far smaller because many invalid branches are eliminated early. When combined with heuristics such as the *minimum remaining values* (MRV) or *forward checking*, The average branching factor k becomes much smaller than 9, resulting in effective performance close to:

$$T_{\text{effective}} \approx O(k^n), \quad k < 9. \tag{7}$$

Table 1 summarizes the main differences between the two search methods.

Table 1: Comparison of Brute-force and Backtracking Search for Sudoku

Aspect	**Brute-force**	**Backtracking**
Search strategy	Blind enumeration	DFS with pruning
Constraint use	None	Enforced at each step
Worst-case $T(n)$	$O(9^n)$	$O(9^n)$
Average performance	Very slow	Efficient in practice
Heuristic support	None	MRV, forward checking

A detailed analysis of how these two algorithms apply to this problem can be found in this work.[5].

1.2 Part 1B

This subsection details our Python-based Sudoku solver. We will start by defining the Sudoku's data structure, then implement the necessary validation functions (`is_valid`, `find_empty`, `is_complete`) and core methods (`set_value`, `clear_value`, `__str__`). The algorithm combines several heuristic and inference techniques[3], including:

- MRV (Minimum Remaining Values): chooses the cell with the fewest legal values to reduce branching.
- LCV (Least Constraining Value): prefers values that least restrict the remaining variables.
- Forward Checking: removes inconsistent values from neighboring domains after each assignment.
- Degree heuristic: if multiple variables tie, selects the one with the largest number of unassigned neighbors.

We only discuss the core concept of our solver, but all these methods are implemented and can be seen in our repository. We will also attach the File Tree in the accessories. We now initialize the solver. Which will read in the Sudoku board and initialize the CSP domain as follows:

```
def _init_domains(self):
    for r in range(9):
        for c in range(9):
            if self.board.grid[r, c] != 0:
                self.domains[(r, c)] = {int(self.
                    board.grid[r, c])}
                self.assigned.add((r, c))
            eElse
                possible = set(range(1, 10))
                for rr, cc in NEIGHBORS[(r, c)]:
                    val = self.board.grid[rr, cc]
                    If val != 0 and val in
                        possible:
                        possible.discard(int(val)
                            )
                self.domains[(r, c)] = possible
```

The solver calls the solve method to calculate the answer and do the metrics. We now go through the _backtrack method, which is a recursive process to solve the Sudoku.

Listing 1: Choose variable

```
def _backtrack(self):
    # goal test
    if len(self.assigned) == 81:
        return True

    var = select_unassigned_variable(self.domains
        , self.assigned,use_mrv=self.use_mrv,
        use_degree=True)
```

We first judge whether this is already a full board. The method `select_unassigned_variable` MRV and Degree heuristic, which will give out the smallest search circumstance as MRV will search each variable's domain-length and find out those which is the smallest so that there will be less values need to try while Degree heuristic will choose a variable with the most unassigned neighbors for reduce total domain size as much as possible, `var` contains which variable will be assigned.

Listing 2: Assign

```
    for value in order_domain_values(var, self.
        domains, use_lcv=self.use_lcv):
        # check consistency quickly using the
            board.is_valid
        r, c = var
        If not self.board.is_valid(r, c, value):
            continue

        # tentatively assign
        self._assign(var, value)
        self.metrics.record_assignment()
```

Method `order_domain_values` uses LCV to choose a value that will be assigned in `var` by visiting `var`'s neighbors and sorting the domains based on how much they will influence the neighbors. If the value is not valid, then we change to the next value.

Listing 3: Forward check and backtrack

```
        # maintain domains: remove value from var
            domain (store old)
        old_domain_var = self.domains[var]
        self.domains[var] = {value}

        inferences = None
        failure = False
        if self.use_fc:
            inferences = forward_checking(self.
                domains, var, value)
            If inferences are None:
                failure = True

        If not failure:
            Result = self._backtrack()
            If result:
                return True

        # undo
        self.metrics.record_backtrack()
        # restore domains
        self.domains[var] = old_domain_var
        if inferences:
            restore_inferences(self.domains,
                inferences)
        self._unassign(var)

    return False)
```

Next, after the assignment, we store the old domains for the cases that require back trace and perform the Forward

Checking. This would change other variables' domains to obey the constraints. If the domain is empty, then this branch can be pruned, as those cases that remain unsearched in this branch can't solve this question. If we can still go through, then we recursively call the `backtrack` method to accept this assignment and try to find the assignment for the next variable. Otherwise, we try another value. If all the values are tried, return false and go back to the last variable to try the next value.When all the variables are assigned, it would cascade return true and finish the search. Below is a process graph for the search.

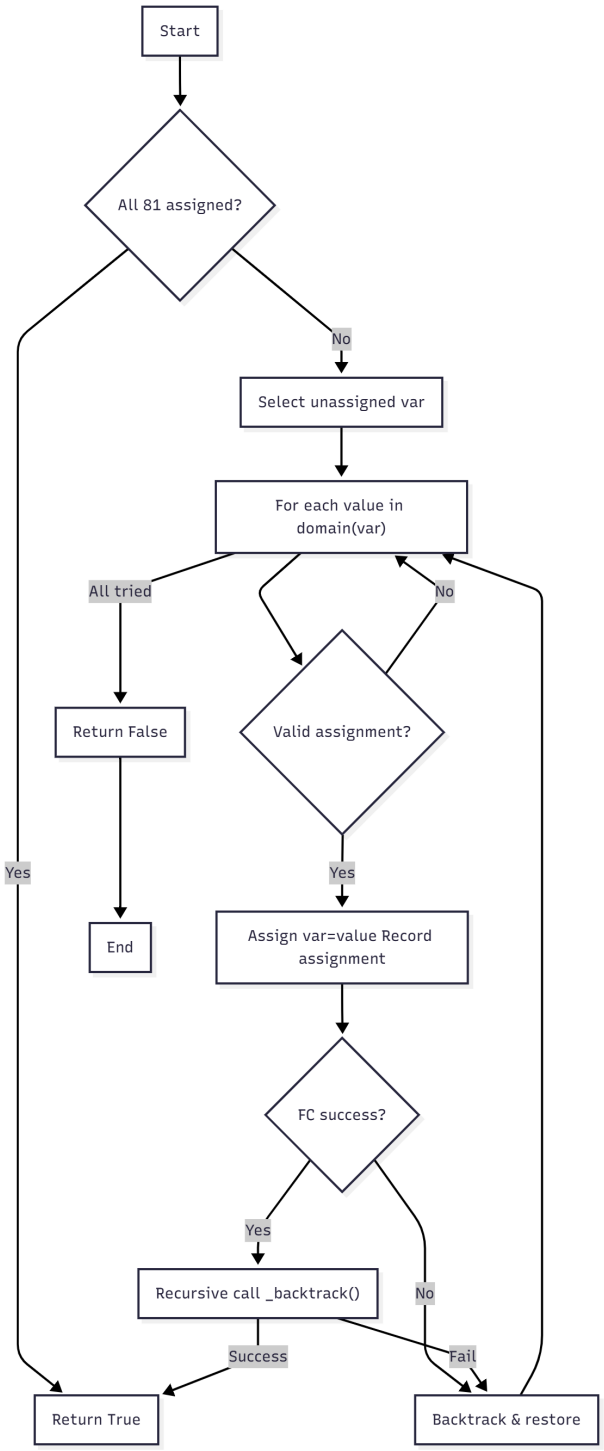


Figure 2: Backtrack.

This solver also do the metrics (assignments, backtracks, total times) and can call `summary` method to print. So the output will be a boolean: whether we find an answer or not; a board data structure which is the answer for the Sudoku; and a metric for this search process.

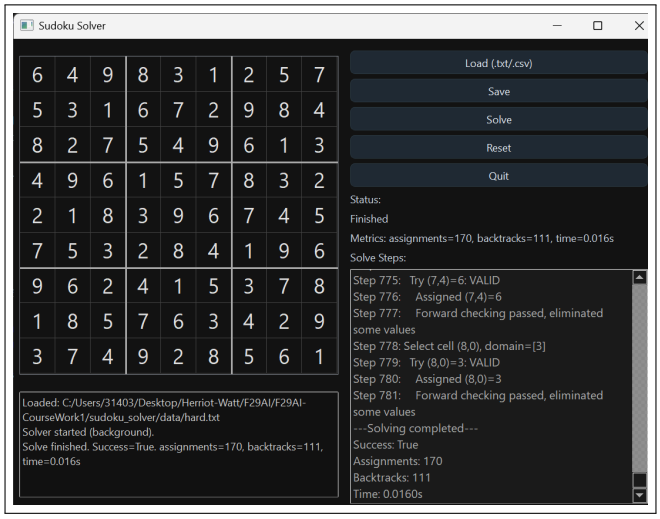


Figure 3: Example interface of the Sudoku solver.

We also implement a GUI for the solver. As shown in Figure **??**, users can load the Sudoku and save the answer. Click the Solve button to start solving, and the "Reset" button to reset the board. The user can also input the numbers into the board by themselves. The chart below will print the solution process, while the right area will print the Metrics for this process.

As Figure A in the appendix shows, we do three tests to solve the core, board, and IO stream. This core test suite verifies the correctness, efficiency, heuristic consistency, and robustness of the Sudoku CSP solver, ensuring it can solve valid puzzles, reject invalid ones, and behave predictably under different configurations. IO test verifies that Sudoku boards can be correctly loaded and saved in both .txt and .csv formats, and that invalid or missing files are properly handled. Board test tests basic functionality of the SudokuBoard class, including validity checking, finding empty cells, and board representation.

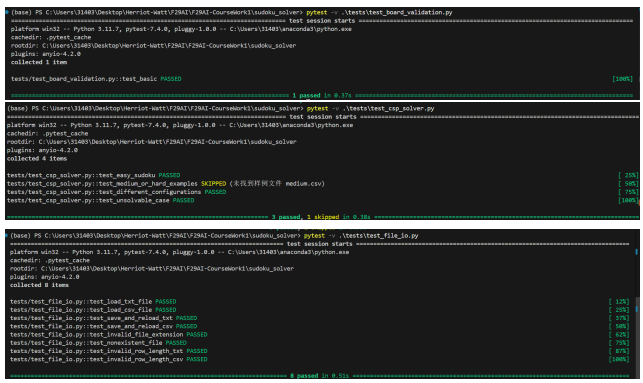


Figure 4: The tests

Now, let's analyze the solving result parameters of Sudoku puzzles with three different difficulty levels. As we can see, the solving time increases follow the Sudoku difficulty increases.

Now we compare the A-star algorithm and our method and explain why the A-star Algorithm is not the best solution for solving the Sudoku.

Table 2: Analyze to 3 solve process

Sudoku	Easy	Medium	Hard
BackTrack	33	105	111
Assignment	90	164	170
Time	9 ms	10ms	11ms

The Model The Sudoku problem is naturally a CSP problem, which is the biggest advantage of using CSP to model it. We don't need to do more transformation to model it in a specific form for our algorithm to solve it.

On the other hand, let's see how A-star performs on this problem. A-star algorithm is finding the lowest-cost path from the starting point to the destination, which is not the goal of the Sudoku: Find any complete chessboard state that satisfies all constraints (i.e., a solution). That means we can't directly construct the model on Sudoku. We need to transform it into a route planning. That makes the case very complex. Then let's consider an implementation of Sudoku. We need to decide the most basic part of A-star.

- Node: A partially filled board. And the set of the filled grids.
- close-set: The state that goes from an empty board to the initial board state. This is because those numbers on the board at the initial state are not allowed to change.
- open-set: Remaining states in the space (Those board states that obey the Sudoku rules) after removing the closed-set.

Just precisely modeling these three elements from a given Sudoku is obviously much more complex than just modeling it as a CSP. This method is neither intuitive nor natural, nor is it easy to implement.

Solve After modeling, the backtrack only needs to assign some heuristic methods. And these heuristic methods are easy to assign (FC, MRV, LCV, etc.). And they can work on the solving process fantastically. As their main job is to prune.

But for A-star, it is highly reliant on the real cost $g(n)$ and the heuristic cost $h(n)$ to search for the best solution. But you can hardly say what is the real cost is in the Sudoku state. And it's also difficult to find a brilliant h(n) that can well guide the search process.

State Space and Memory The state of the backtrack method is reducing. This is because while the process goes further, it will keep pruning the mistake branch and reduce the size of the domain. In other words, it performs a subtraction.

For A-star, the case becomes much worse. It needs to maintain open-set and close-set to determine subsequent searches, and then it is also difficult for it to find an excellent heuristic. This leads to a high probability that it will directly retreat to the approximate Dijstra algorithm. What's worse, it's even hard to give a proper definition of the cost between each state. It's more like a purpose-free breadth-first search. This is fatal in Sudoku problem-solving, manifested as almost inefficient search and an extremely large search space and memory requirement.

In short, choose A-star for Sudoku solving isn't a patch on the backtrack method. The table concludes the comparison of these two methods.

Feature	Backtracking	A* Algorithm
Modeling	Natural CSP formulation	Transform to path-search
Goal	Find a valid solution	Find minimum-cost path
Cost	Not required	meaningful costs
Heuristic	Pruning (MRV, LCV, FC)	effective / admissible $h(n)$
Strategy	DFS backtracking	May degrade to Dijkstra / BFS
Search Space	Small via propagation	Huge when $h(n)$ weak
Memory	Low	Huge (open/close sets)

Table 3: Condensed comparison

2 Automated Planning

In this section, we will try to implement an automated planning for a lunar rover to collect the data and samples on the moon. We use the PDDL language to implement this. Still, we introduce it with three subsections. Part 2A will detail the domain file of the task, clearly describe a closed knowledge base, and the actions of the whole task system. Part 2B will show how the solver automates planning two different missions through the domain defined in Part 2A. Yet the Part 2C will do the extension on this task. All of our plan is compiled by Lama-First, and you can see the result at appendix.

2.1 Part2A & 2B

In this section, we focus on giving out a domain for this mission. We need to first give out the type of objects we have on this mission: **Lander**; **Rover**; **Location**; **Sample**; **Scan**; **Image**. Because the lander and the rover are all kinds of **Equipment**, their father is an equipment type. Similarly, the scan and image are kind of **Data**.

After we confirm all the types of objects, we define each object's predicates.

- is_landed: The function performed by this predicate is to check whether the lander has landed.
- on_position: This predicate is used to check whether the equipment is at the target location.
- is_connected: This predicate is used to check whether two target locations are reachable.
- need_image, need_sample, need_scan: These three predicates all operate on the location and respectively indicate whether the target location requires taking photos, sampling or scanning.
- captured_data: This predicate indicates whether the data collection task has been completed.

- collected_sample: Consequently, this predicate is used to indicate whether the sample collection task has been completed.
- empty_memory: This predicate checks whether the memory of the rover is empty.
- hand_empty: This predicate checks whether the hand of the rover is empty.
- is_full: This predicate indicates whether the sample compartment of the lander is full.
- has_received: This predicate checks whether the data transmission has been completed.
- sample_got: This predicate indicates whether the sample has been obtained.
- association: This predicate indicates whether the lander and the rover are properly paired.

Through the task description, we designed the following actions.

- lander_land: The main purpose of this operation is to land the lander.

  To achieve this action, we need three elements first: the lander, the rover, and the location.

  Secondly, what we need to obtain is an unlanded lander, so that we can commence this operation.

  Finally, the goal we aim to achieve is for the lander to land precisely at the designated spot, and at the same time, for the rover to reach the prescribed location as well. Once all these tasks are completed, the operation will come to an end.

- rover_move: This action is to move the rover from the starting location to the destination location.

  Therefore, we obviously need to prepare a rover, and determine the starting location A and the destination location B for the two locations.

  Before carrying out the operation, we need to confirm three things: the rover is at A, not at B, and location A and B are connected.

  Then we can carry out the task, making the rover move from location A to location B.

- take_picture: The purpose of this action is to take pictures at a specific location.So the things we need to cover are the rover, the location, and the image.

  First, we need to confirm whether the rover is at the correct location and whether photos need to be taken at that location. Secondly, we need to check if the memory is empty (that is, if there is sufficient space for taking photos).

  Once all the conditions are met, we will commence this operation. What needs to be settled is that after the photos are taken, the destination location should be set as no longer requiring photography, and the onboard memory of the rover should be occupied.

- take_scan: Just like the previous operation, this mission has changed from taking photos to scanning the area. Everything else remains the same.
- take_sample: Unlike the previous two actions, the process of collecting samples did not utilize memory. Instead, it was accomplished by using hands for collection. Other things remain the same.
- transmit_data: This action is aimed at transmitting data. Therefore, the objects we need to operate on are the lander, the rover, and the data.

  Before completing this operation, we need to ensure that there are untransmitted data on the rover, and that the rover is compatible with the target lander.

  After completion, the memory storing the data needs to be cleared. At this point, the lander is in the state of having retrieved the data.

- sample_receive: This operation is to transfer the samples collected by the rover back into the lander. Therefore, we need the lander, the rover, and the sample exchange location.

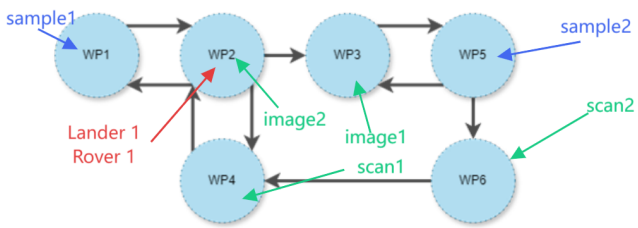
  In order to complete this action, the lander and the lunar rover should be at the same location, and the rover driver should be holding the collected samples.

  After meeting these conditions, the rover will hand over the samples to the lander. The lander will then become fully loaded. The rover will then be free to carry out its mission.

All of this is described at the Figure 6.

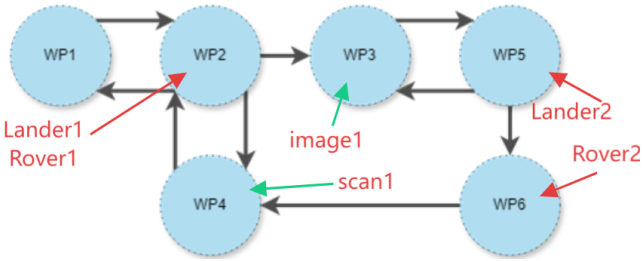
What I want to do next is try to make an analysis on mission 2. We would first check the map.

Figure 5: The map for mission 2



We now have multiple choice,s like land the second lander or rover1 take the image2 or rover move. These actions' predicates are all satisfied. The solver decide to let the rover take the sample. So rover1 moves to the WP1 and the second lander and rover deploy at the WP5. Now "take sample" action is prepared for both rovers. After taking a sample from the WP1 and WP5, the state changes. To achieve the state recieve the sample rover1 needs to move back to WP2. Then two rovers can give the lander their sample.

But note that our rover2 can submit its sample earlier, so rover2 moves to WP6 after submitting then take the scan. Later rover1 move back to lander to submit and take the image. Now the state update. Through the state and the predicate, actually the followed actions are easy to predict. The rover2 can only move to WP4 to take scan, and the best choice for rover1 is to go to WP3 to take the image. This is



also how the plan in the appendix describes. Also, notice that because of the "memory empty" predicate the rover needs to transmit the data first, then collect it.

2.2 Part2C

In order to complete the task of part 2C, we need to make appropriate changes to the previous domain. The following are the revisions regarding the new field:

- New objects: human, astronaut, internal_area, control_room - internal_area, docking_bay - internal_area.
- New predicate-is_deployed: Confirm whether the rover has been deployed.
- New predicate-is_area_of - internal_area: Used to confirm whether it is the internal area of the target lander.
- New predicate-astronaut_at ?a - astronaut ?l - lander ?ia - internal_area: Used to confirm whether the astronaut is within the internal area of the target lander.
- New action-astronaut_move_internal: This action enabled the astronaut to move from one area to another.
- New action-rover_deploy: This action is to enable the astronauts to deploy the rover.
- Action update-lander_land: Remove the deployment of the lunar rover and proceed independently with another operation.
- Action update-transmit_data, sample_receive: Added the movements of the astronauts.

Notice we separate the deployment and the land. Because the predicate is different: Lander landing don't require the astronaut at the docking bay, while deployment needs. The other changes are clear and understandable, except "is area of". This action is to ensure the astronauts will not teleport to another lander.

One more thing to remind is that the planer will give out the different plan while using different compiler. For example on mission 2. If we use the BFWS-FF then the rover will take most of the job while the compiler would prefer to do the data collecting first.

3 Video

In the end our video is in the OneDrive

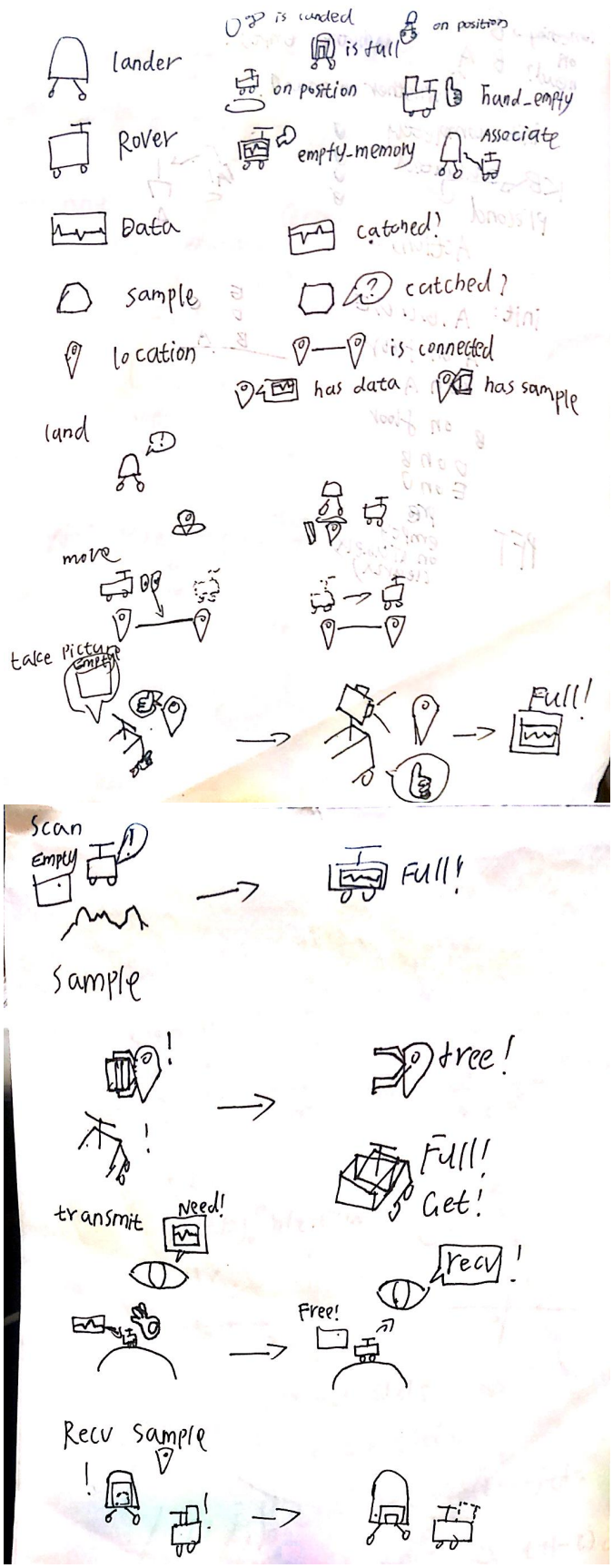


Figure 6: A simple explanation

References

[1] U. Montanari, "Networks of constraints: Fundamental properties and applications to picture processing," *Information sciences*, vol. 7, pp. 95–132, 1974.

[2] I. Howell, R. J. Woodward, B. Y. Choueiry, and C. Bessiere, "Solving sudoku with consistency: A visual and interactive approach.," in *IJCAI*, vol. 2018, pp. 5829–5831, 2018.

[3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. USA: Prentice Hall Press, 3rd ed., 2009.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms, Third Edition*. The MIT Press, 3rd ed., 2009.

[5] B. V. Indriyono, R. Wardani, T. Susanti, L. Wulandari, M. Fathurrohim, R. S. Pratiwi, and E. W. Safitri, "Analysis of the concept of solving the sudoku game using backtracking and brute force algorithms," *International Journal of Artificial Intelligence & Robotics (IJAIR)*, 2024.

A File Tree

```
sudoku_solver/ ..........................................Project root directory
├── main.py ..............................Program entry: launch GUI
├── requirements.txt ....................List of required libraries (PyQt5, NumPy, etc.)
├── README.md ...........................Brief description (usage, features, screenshots, etc.)
├── data/ ...............................Puzzle datasets
│   ├── easy.txt
│   ├── medium.csv
│   └── hard.txt
├── sudoku_core/ ........................CSP solver core modules
│   ├── __init__.py
│   ├── board.py ........................Grid management and validity checking
│   ├── csp_solver.py ...................Backtracking + MRV + LCV + Forward Checking
│   ├── heuristics.py ...................Heuristic functions (MRV, LCV, Degree heuristic)
│   ├── inference.py ....................Inference algorithms (Forward Checking / AC-3)
│   └── metrics.py ......................Performance metrics (time, backtracks, recursion depth)
├── gui/ ................................Graphical user interface
│   ├── __init__.py
│   ├── main_window.py ..................Main window (QMainWindow)
│   ├── board_widget.py .................Sudoku board widget (editable grid)
│   ├── control_panel.py ................Control panel (load, solve, reset, show metrics)
│   ├── visualizer.py ...................Step-by-step animation logic
│   └── styles.qss ......................GUI stylesheet (theme and appearance)
├── utils/ ..............................Utility modules
│   ├── __init__.py
│   ├── file_io.py ......................Read/write .txt or .csv files
│   ├── timer.py ........................Performance timing decorator
│   └── logger.py .......................Logging utilities (debug and event output)
└── tests/ ..............................Unit tests
    ├── test_csp_solver.py ..............Unit test for CSP solver
    ├── test_board_validation.py ........Unit test for board validation
    └── test_file_io.py .................Unit test for file I/O
```

Figure 7: Project File Tree Overview (dirtree format)

B Plan Result

```
Plan 1:
;;!domain: lunar
;;!problem: lunar-mission-1

0.00000: (lander_land lander rover wp1)
0.00100: (take_sample rover wp1 sample)
0.00200: (sample_receive lander rover sample wp1)
0.00300: (rover_move wp1 wp2 rover)
0.00400: (rover_move wp2 wp3 rover)
0.00500: (take_scan rover wp3 scan)
0.00600: (rover_move wp3 wp5 rover)
0.00700: (transmit_data rover scan lander)
0.00800: (take_picture rover wp5 image)
0.00900: (transmit_data rover image lander)
```

```
Plan2:
;;!domain: lunar
;;!problem: lunar-mission-2

0.00000: (rover_move wp2 wp1 rover1)
0.00100: (take_sample rover1 wp1 sample2)
0.00200: (lander_land lander2 rover2 wp5)
0.00300: (take_sample rover2 wp5 sample1)
0.00400: (sample_receive lander2 rover2 sample1
    wp5)
0.00500: (rover_move wp5 wp6 rover2)
0.00600: (take_scan rover2 wp6 scan2)
0.00700: (rover_move wp1 wp2 rover1)
0.00800: (sample_receive lander1 rover1 sample2
    wp2)
0.00900: (take_picture rover1 wp2 image2)
0.01000: (rover_move wp6 wp4 rover2)
0.01100: (rover_move wp2 wp3 rover1)
0.01200: (transmit_data rover1 image2)
0.01300: (take_picture rover1 wp3 image1)
0.01400: (transmit_data rover1 image1)
0.01500: (transmit_data rover2 scan2)
0.01600: (take_scan rover2 wp4 scan1)
0.01700: (transmit_data rover2 scan1)
```

```
Plan 3
;;!domain: lunar-extended
;;!problem: lunar-mission-3

0.00000: (rover_move wp2 wp1 rover1)
0.00100: (take_sample rover1 wp1 sample2)
0.00200: (rover_move wp1 wp2 rover1)
0.00300: (rover_move wp2 wp4 rover1)
0.00400: (take_scan rover1 wp4 scan1)
0.00500: (lander_land lander2 wp6)
0.00600: (rover_move wp4 wp2 rover1)
0.00700: (astronaut_move_internal alice lander1
    docking_bay1 control_room1)
0.00800: (transmit_data rover1 scan1 lander1
    alice control_room1)
0.00900: (rover_move wp2 wp3 rover1)
0.01000: (take_picture rover1 wp3 image1)
0.01100: (rover_move wp3 wp5 rover1)
0.01200: (transmit_data rover1 image1 lander1
    alice control_room1)
0.01300: (rover_move wp5 wp6 rover1)
0.01400: (take_scan rover1 wp6 scan2)
0.01500: (rover_move wp6 wp4 rover1)
0.01600: (rover_move wp4 wp2 rover1)
0.01700: (transmit_data rover1 scan2 lander1
    alice control_room1)
0.01800: (take_picture rover1 wp2 image2)
0.01900: (transmit_data rover1 image2 lander1
    alice control_room1)
0.02000: (astronaut_move_internal alice lander1
    control_room1 docking_bay1)
0.02100: (sample_receive lander1 rover1 sample2
    wp2 alice docking_bay1)
0.02200: (astronaut_move_internal bob lander2
    control_room2 docking_bay2)
0.02300: (rover_deploy rover2 lander2 wp6 bob
    docking_bay2)
0.02400: (rover_move wp6 wp4 rover2)
0.02500: (rover_move wp4 wp2 rover2)
0.02600: (rover_move wp2 wp3 rover2)
0.02700: (rover_move wp3 wp5 rover2)
0.02800: (take_sample rover2 wp5 sample1)
0.02900: (rover_move wp5 wp6 rover2)
0.03000: (sample_receive lander2 rover2 sample1
    wp6 bob docking_bay2)
```